Table des matières

Cours 03 — Monter le projet dbt et le STAGING 1
Objectifs 1
1. Ce qu'est un modèle, et ce que dbt ajoute 2
2. Le projet et sa configuration 2
3. La connexion, hors du dépôt 3
4. Les sources 3
5. Où dbt écrit ce qu'il produit 4
6. Les quatre gestes du `STAGING` 4
7. L'état courant, et ce qu'il répare 5
8. Les matérialisations 5
9. `ref()` et le graphe 6
10. Monter le projet, en pratique 6
À retenir 6
Exercices 7
Annexe A — Le détail des étapes du labo 7

Cours 03 — Monter le projet dbt et le STAGING

Théorie · Cours 3/9 — Formation Analytics Engineering · intermédiaire

Où vous êtes : `RAW` → **`STAGING`** → `INTERMEDIATE` → `MARTS`. Le cours ouvre l'outil du métier et bâtit la première couche, celle qui rend chaque source lisible sans encore rien joindre.

Objectifs

- Dire ce qu'est un modèle dbt, et ce que dbt ajoute autour d'un `select`.
- Monter un projet dbt dans un dépôt existant, et dire où vit sa connexion et pourquoi.
- Déclarer une source, et la lire par `source()`.

© 2026 iAlexMG. Tous droits réservés.

- Nommer les quatre gestes du `STAGING`, et les deux qu'il refuse.
- Tirer l'état courant d'une table livrée par lots, et dire ce qu'il répare au passage.
- Choisir une matérialisation, et dire où dbt écrit ce qu'il bâtit.
- Relier deux modèles par `ref()`, et lire le graphe qui en résulte.

---

1. Ce qu'est un modèle, et ce que dbt ajoute

Un modèle dbt est un fichier `.sql` qui contient un `select`, et rien d'autre : ni `create`, ni `insert`, ni point-virgule final. Le nom du fichier devient le nom de l'objet, et dbt écrit le `create` autour.

Ce que dbt ajoute	Ce que ça change
un fichier par modèle, dans un dépôt Git	une transformation se relit, se compare, se révise
un graphe de dépendances, déduit du code	l'ordre d'exécution n'est plus tenu à la main
des tests déclarés à côté du modèle (Cours 05)	une règle se vérifie à chaque construction
une documentation tirée du code (Cours 05)	elle ne peut pas dériver de ce qu'elle décrit

- dbt n'est pas un moteur : il compile votre SQL, l'envoie à DuckDB et lit la réponse. Ce qui est lent dans un projet dbt est lent dans la base, et se répare dans la base.
- Le SQL du Labo 02 était juste, et il a disparu avec le notebook. Celui-ci reste, versionné.

2. Le projet et sa configuration

Un projet dbt est un dossier qui contient un `dbt_project.yml` : c'est ce fichier qui le fait exister. Le dossier, versionné par Git, range ses fichiers par sorte :

- un seul fichier de projet, `dbt_project.yml`, qui règle le projet entier ;
- sous `models/`, les modèles, un fichier `.sql` par vue ou par table, et les fichiers YAML qui les décrivent, comme `_sources.yml` ;
- sous `macros/`, les macros, du Jinja que dbt exécute, comme `generate_schema_name` ;
- plus tard, sous `tests/`, des tests, au Cours 05.

La connexion à l'entrepôt n'en fait pas partie : elle vit dans un second fichier YAML, `profiles.yml`, hors du dépôt (section 3).

© 2026 iAlexMG. Tous droits réservés.

Ce que `dbt_project.yml` déclare	À quoi ça sert
`name`	le nom du projet, repris dans les configurations
`profile`	le profil de connexion à lire dans `profiles.yml`
`model-paths`, `macro-paths`	où chercher les fichiers
le bloc `models:`	les réglages par défaut, dossier par dossier

- Les réglages du bloc `models:` descendent en cascade, du projet vers le dossier, puis vers le fichier : `+materialized: view` posé sur `staging` vaut pour tous ses modèles, et un modèle qui déclare autre chose l'emporte. On le dit une fois, sur le dossier.

3. La connexion, hors du dépôt

Le `dbt_project.yml` ne dit pas comment se connecter : c'est `profiles.yml` qui le dit, et il vit hors du dépôt, parce qu'il porte de quoi se connecter.

Où dbt cherche `profiles.yml`, dans l'ordre	Chez Boréal
à l'endroit que désigne `DBT_PROFILES_DIR`, si la variable est posée	l'intégration continue, au Cours 08
sinon, dans le dossier d'où on le lance	`Projet_Boreal_intermediaire`, toujours
sinon, dans le dossier `.dbt` du répertoire personnel	rien

- Sous DuckDB, le profil ne porte qu'un chemin ; sur une plateforme infonuagique, il porte un compte et de quoi s'authentifier. La règle vaut donc partout, pour ne pas avoir à s'en souvenir le jour où elle compte.
- Un chemin relatif se lit depuis le dossier d'où l'on lance dbt, pas depuis le projet : dbt se lance donc toujours depuis `Projet_Boreal_intermediaire`, et `DBT_PROJECT_DIR` lui dit que son projet est dans `02_ae`.

Piège — un `profiles.yml` vide est pire qu'absent. Sans fichier, dbt dit où il a cherché. Avec un fichier vide, dbt 1.12.3 s'arrête sur `cannot access local variable 'summary_message'`, qui ne dit rien à personne.

4. Les sources

Une source est une table que vous n'avez pas fabriquée et que vous vous engagez à ne pas modifier : chez Boréal, les tables de la `RAW`. Un modèle la lit par `{{ source('raw', 'erp_kna1') }}`, jamais par son nom. Les doubles accolades sont du Jinja, le langage de gabarits de dbt : avant d'envoyer la requête, dbt les remplace par le nom complet de la table, `"boreal"."raw"."erp_kna1"`. Le Jinja n'a pas de fichier à lui : il s'écrit dans le modèle même, au milieu du SQL, et la macro de l'étape 3 du labo en est faite tout entière.

© 2026 iAlexMG. Tous droits réservés.

Ce qu'on gagne à la déclarer	Concrètement
le graphe remonte jusqu'à la source	on voit ce qui casse si elle change
le nom du schéma vit à un seul endroit	changer d'environnement ne réécrit aucun modèle
la documentation la connaît	elle paraît dans le lignage, avec sa description
sa fraîcheur devient vérifiable	dbt peut dire si elle n'a pas été alimentée, au Cours 08

- Une source se déclare le jour où un modèle la lit : une source déclarée pour rien est une promesse que rien ne vérifie.

5. Où dbt écrit ce qu'il produit

- Par défaut, un schéma déclaré dans le projet s'ajoute à celui du profil au lieu de le remplacer : un profil qui vise `staging` et un dossier qui déclare `+schema: staging` écrivent dans `staging_staging`. Sur une plateforme partagée, cela évite que deux développeurs écrivent au même endroit ; ici, cela donne des noms que personne n'a voulus.
- La macro `generate_schema_name` dit à dbt de prendre le nom déclaré tel quel. Tout projet finit par l'écrire ; celle-ci ne changera qu'au Cours 07, quand le projet aura deux environnements.

6. Les quatre gestes du `STAGING`

La `RAW` garde les fichiers tels que livrés, et le Cours 01 en a vu les pièges : tout y est en texte, si bien que `9996.26` passe devant `293478.95` ; une commande y revient à chaque livraison ; des codes comme `01` ou `X` sont à deviner. Le `STAGING` en fait des tables qu'on interroge sans piège : une vue par table livrée, et quatre gestes.

Le geste	Ce qu'il fait chez Boréal
Renommer	`KUNNR` devient `code_concessionnaire`, `NETWR` devient `valeur_nette`
Typer	la date SAP `20230302` devient une date, le montant en texte un décimal
Décoder	le code `01` devient Or, d'après le dictionnaire de l'univers
Tirer l'état courant	des livraisons d'une commande, garder la plus récente

- Le quatrième est celui qu'on oublie : sans lui, les 26 538 lignes de `raw.erp_vbak` passent pour 25 483 commandes, et chaque total de la chaîne est faux de quelques pour cent, sans un message.
- Le `STAGING` refuse deux gestes :

© 2026 iAlexMG. Tous droits réservés.

    - retirer des lignes par un `where`, parce qu'elles ne compteraient pas pour le métier : les quatre concessionnaires à la fiche fermée restent dans la vue, marqués par `est_supprime`, et les postes refusés aussi, que l'`INTERMEDIATE` écarte au Cours 04. Tirer l'état courant ne retire que des copies : chaque commande reste, une seule fois ;
    - joindre une autre table : une jointure est une décision de modélisation, et la modélisation a ses couches. Au staging, décoder une colonne oui, joindre une autre table non.

- Le `STAGING` et le stage de Snowflake se ressemblent par le nom, rien d'autre : le stage est un emplacement de fichiers, où l'ingénieur de données dépose un CSV avant de le charger dans la `RAW`.

7. L'état courant, et ce qu'il répare

Tirer l'état courant, c'est numéroter les livraisons de chaque clé, de la plus récente à la plus ancienne, puis ne garder que la première. Deux outils de SQL le font, l'un après l'autre :

- `row_number() over (partition by kunnr order by _lot desc) as rang` numérote. Il repart à 1 pour chaque valeur de la colonne de `partition by`, ici pour chaque concessionnaire, et suit l'ordre de la colonne de `order by`, ici du lot le plus récent au plus ancien.
- `where rang = 1` garde. Posé sur le résultat du `with`, il ne retient que les lignes numérotées 1.

Le lot le plus récent est le plus grand : `_lot` dit le mois, et `2024-06` vient après `2024-05`. `row_number()` s'apprend au Cours 04 de la formation SQL avancé, `Theorie_04_SQL.md`, section 3, et `with` à son Cours 05, `Theorie_05_SQL.md`, section 2.

- Il répare au passage l'histoire. Chez Boréal, une commande livrée au lot de mai 2024 pointe un concessionnaire dont la fiche n'arrive qu'en juin : à l'époque, elle était orpheline ; sur l'état courant, elle ne l'est plus, et le contrôle rend zéro.
- Ce zéro est un choix : regarder le référentiel d'aujourd'hui plutôt que celui de l'époque. Le jour où la dimension passera en type 2, au Cours 05, la même question donnera une autre réponse, et les deux seront justes.

8. Les matérialisations

Matérialisation	**Ce que dbt écrit**	**Quand la prendre**
`view`	une vue : la requête rejouée à chaque lecture	le `STAGING`, et ce qui est lu rarement
`table`	une table : le résultat écrit une fois	les `MARTS`, lus souvent et par plusieurs
`ephemeral`	rien : le SQL est injecté dans les modèles qui l'appellent	un calcul intermédiaire, au Cours 04

© 2026 iAlexMG. Tous droits réservés.

Matérialisation	Ce que dbt écrit	Quand la prendre
`incremental`	une table complétée par tranches	un gros fait qui s'allonge, au Cours 07

- `view` est ce que dbt bâtit quand rien n'est dit. Le `STAGING` en vues est le choix courant, celui que recommande dbt Labs, l'éditeur de dbt : une vue ne coûte rien à bâtir, elle est toujours à jour, et renommer ou typer coûte peu à la lecture. Une équipe le passe en tables le jour où relire la `RAW` à chaque requête coûte trop cher. Les vues s'apprennent au Cours 03 de la formation SQL avancé, `Theorie_03_SQL.md`, section 7.
- Le préfixe dit la couche, pas la forme : `stg_concessionnaires` resterait `stg_` en table.

9. `ref()` et le graphe

Un modèle qui en lit un autre écrit `{{ ref('stg_commandes') }}` : dbt met le nom réel de l'objet, et surtout il enregistre le lien.

- De ces liens, dbt tire l'ordre d'exécution, ce qu'il faut rebâtir quand un modèle change, et le lignage de la documentation.
- `dbt ls` liste ce que dbt connaît : la première commande à lancer quand un chiffre surprend.

Piège — un modèle qui nomme un autre modèle en dur fonctionne, et casse le graphe : dbt ne voit pas le lien, peut bâtir les deux dans le mauvais ordre, et le lignage ment. `source()` pour ce qui vient d'ailleurs, `ref()` pour tout ce que le projet fabrique.

10. Monter le projet, en pratique

Question	Ce qu'on fait	Chez Boréal (labo)
dbt trouve-t-il le projet et la connexion ?	`dbt debug`	`All checks passed!` (1.1, 2.3)
Où dbt écrira-t-il ?	la macro des schémas, puis le nom constaté	`staging`, pas `staging_staging` (3.1, 6.3)
Que connaît dbt ?	déclarer les sources, puis `dbt ls`	deux sources, puis trois, et quatre modèles (4.1, 5.1, 8.3)
Le décodage a-t-il pris ?	compter ce qui reste vide après le `case`	0 palier vide, 0 gamme vide (6.5, 6.6)
L'état courant tient-il ?	la `RAW` contre la vue	26 538 lignes, 25 483 commandes (7.4)
Les vues se tiennent-elles entre elles ?	un contrôle relié par `ref()`	0 commande sans concessionnaire (8.4, 8.5)

À retenir

© 2026 iAlexMG. Tous droits réservés.

- Un modèle dbt est un `select` dans un fichier. dbt écrit le `create`, et exécute dans l'ordre que le graphe dicte.
- `dbt_project.yml` fait exister le projet ; `profiles.yml` dit comment se connecter, et vit hors du dépôt.
- Une source se déclare avant de se lire ; `ref()` relie les modèles. Nommer en dur fonctionne et casse tout ce que dbt sait faire.
- Le `STAGING` renomme, type, décode et tire l'état courant. Il ne retire aucune ligne du métier et ne joint rien.
- L'état courant fait disparaître les doublons de livraison, et répare au passage l'histoire : c'est un choix.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_03_AE.md`. Faites-les après le labo : plusieurs énoncés supposent que le projet tourne.

Annexe A — Le détail des étapes du labo

A.1 Étape 1 : la connexion

Labo : `Labo_03_AE.md`, étape 1.

Notions : section 3, la connexion hors du dépôt.

- 1.1. Le nom `boreal_ae` est celui du profil, que `dbt_project.yml` réclame à l'identique. Le chemin se lit depuis `Projet_Boreal_intermediaire`, et le nom du fichier, `boreal`, devient celui de la base pour dbt : celui sous lequel le notebook l'attache. L'entrepôt, le fichier `boreal.duckdb` et la base `boreal` sont un seul objet, vu de trois côtés (Théorie 01, annexe A.1). `schema: staging` est le schéma par défaut de la cible ; `threads: 4` laisse dbt bâtir quatre modèles à la fois quand le graphe le permet.

A.2 Étape 2 : le projet

Labo : `Labo_03_AE.md`, étape 2.

Notions : section 2, le projet et sa cascade.

- 2.1. Le dépôt et le projet sont la même chose : le projet vit à la racine de `02_ae`. `dbt init`, qui crée un dossier et son squelette, ne convient pas ici : il créerait un sous-dossier, et un nom de projet dbt ne peut pas commencer par un chiffre, `dbt init 02_ae` restant bloqué dans une invite qui redemande un nom.

© 2026 iAlexMG. Tous droits réservés.

- 2.3. `dbt debug` vérifie les deux fichiers, puis ouvre l'entrepôt. Il ne bâtit rien.

A.3 Étape 3 : la macro des schémas

Labo : `Labo_03_AE.md`, étape 3.

Notions : section 5, où dbt écrit.

- 3.1. Si un modèle déclare un schéma, la macro le prend tel quel ; sinon, elle prend celui du profil. Dans un fichier de Jinja pure, un commentaire s'écrit `{# … #}` et non `--` : dbt évalue ce qui ressemble à du Jinja même dans un commentaire SQL.

A.4 Étape 4 : les sources

Labo : `Labo_03_AE.md`, étape 4.

Notions : section 4, les sources.

- 4.1. Deux tables, celles que les premières vues lisent. Les articles arrivent à l'étape 6, les postes de commande au Cours 04 ; le schéma `argent` de l'usine ne sert qu'à Boréal avancé.

A.5 Étape 5 : ce que dbt connaît

Labo : `Labo_03_AE.md`, étape 5.

Notions : section 4, les sources ; section 9, le graphe.

- 5.1. Aucun modèle encore. L'avertissement vise le bloc `staging` du projet, que rien ne remplit : il disparaît à l'étape 6.

A.6 Étape 6 : les premières vues

Labo : `Labo_03_AE.md`, étape 6.

Notions : section 6, les quatre gestes ; section 8, la vue.

- 6.1. Les quatre gestes, et rien d'autre : renommer dans le `select` ; typer par `try_strptime` et par la comparaison qui rend un booléen ; décoder par le `case` ; tirer l'état courant par `row_number()` et `where rang = 1`. Aucune jointure. Une colonne de texte, comme `name1` ou `ort01`, se renomme sans se convertir : la `RAW` la garde déjà en texte, et la vue la reprend telle quelle.
- 6.2. Le même geste sur les articles, que le prototype du Cours 02 avait déjà décodés. La vue servira à `dim_produit`, au Cours 04.
- 6.3. dbt bâtit les deux vues en parallèle : rien ne les relie.
- 6.4. Le notebook attache l'entrepôt sous `boreal`, le nom que dbt lui donne. Il lit ce que dbt a écrit, dans le schéma `staging`, et n'écrit rien.

© 2026 iAlexMG. Tous droits réservés.

- 6.5 et 6.6. Le contrôle du décodage, celui du Cours 02 : un `case` qui ne trouve rien rend `NULL` en silence. Quatre concessionnaires ont une fiche fermée, et restent au staging, marqués par `est_supprime` : décider lesquels comptent revient aux couches suivantes (section 6). Les 23 articles sans année-modèle sont les pièces.
- 6.7. Sous Windows, un fichier DuckDB tenu par un processus refuse l'écriture des autres, même quand il n'est ouvert qu'en lecture : détacher libère l'entrepôt sans fermer le notebook.

A.7 Étape 7 : les 25 483 commandes

Labo : `Labo_03_AE.md`, étape 7.

Notions : section 6, l'état courant ; section 7, ce qu'il répare.

- 7.1. `nullif` fait de la date vide de SAP une absence : on veut dire « pas de modification », et non « conversion ratée ». `netwr` se type en `decimal`, jamais en `double` : un montant se compte au cent près.
- 7.4. Les 1 055 lignes en trop de la `RAW` ont disparu, et aucune commande n'est perdue. 2 231 commandes portent une date de modification.

A.8 Étape 8 : le graphe

Labo : `Labo_03_AE.md`, étape 8.

Notions : section 7, l'orpheline réparée ; section 9, `ref()` et le graphe.

- 8.1. Le contrôle joint deux vues, ce que le staging refuse : il n'est pas un modèle de la couche, mais un contrôle, et son préfixe `ctl_` le dit. Personne ne le consomme.
- 8.2. dbt bâtit le contrôle après les deux vues, parce que `ref()` lui a dit qu'il en dépendait. Vous ne le lui avez dit nulle part ailleurs.
- 8.5. La commande `0010015405`, livrée au lot de mai 2024, pointe le concessionnaire `0000200154`, dont la première fiche arrive au lot de juin : l'orpheline d'hier, que l'état courant a réparée.
- 8.7. Premier travail du dépôt : le projet, la macro et les modèles. `profiles.yml` reste dehors, et `target/` et `logs/`, que dbt refait, sont écartés par le `.gitignore` du Cours 01.

© 2026 iAlexMG. Tous droits réservés.